SCOPE: Sequential Causal Optimization of Process Interventions

Abstract. Prescriptive Process Monitoring (PresPM) recommends interventions during running business processes to optimize key performance indicators (KPIs). In realistic settings, interventions are rarely isolated: organizations need to align sequences of interventions to jointly steer the outcome of a case. Existing PresPM approaches only partially address this challenge. Many focus on a single intervention decision, while others treat multiple interventions independently, ignoring how they interact over time. Methods that do address these dependencies depend either on simulation or data augmentation to approximate the process to train a Reinforcement Learning (RL) agent, which may create a reality gap and introduce bias. We introduce *SCOPE*, a PresPM approach that learns aligned sequential intervention recommendations. *SCOPE* employs backward induction to estimate the effect of each candidate intervention action, propagating its impact from the final decision point back to the first. By leveraging causal learners, our method can utilize observational data directly, unlike methods that require constructing process approximations for reinforcement learning. Experiments on both an existing synthetic dataset and a new semi-synthetic dataset show that *SCOPE* consistently outperforms state-of-the-art PresPM techniques in optimizing the KPI. The novel semi-synthetic setup, based on a real-life event log, is provided as a reusable benchmark for future work on sequential PresPM.

Keywords: Prescriptive Process Monitoring · Sequential Decision-Making · Causal Learning

1 Introduction

PresPM uses machine learning to provide case-specific recommendations at different decision points during the execution of business processes. These recommendations concern interventions, such as managerial escalations or customer communications, that aim to improve KPIs, for example, throughput time or cost efficiency. PresPM holds the potential to move organizations from merely describing and predicting process behavior towards actively steering process executions [8]. In this paper, we use the terms *intervention* for any controllable action taken on an ongoing case, and *intervention recommendation* for the action suggested by a PresPM method at a *decision point*.

In many processes, intervention decisions are not isolated. Typically, a process contains multiple, interdependent decision points that jointly determine the outcome of a case. The effect of an earlier intervention depends on which interventions will be applied later in the same case. Optimizing intervention decisions one by one is therefore insufficient, because actions that look beneficial locally can undermine overall KPI performance. For example, in a marketing process aimed at maximizing revenue, the effectiveness of offering a client a discount may depend on an earlier choice to send a promotional email and the client's response to that email. This sequence

of decisions together shapes the revenue. Optimizing each decision independently without considering the future might make it look like sending a promotional email is only moderately valuable for some clients, even though if you follow it with a discount, it could be highly effective. Methods for PresPM thus need to reason over sequences of decisions and their combined effect on the final KPI.

Most existing PresPM approaches do not yet offer such sequential support. First, many methods address only a single intervention scenario, even when multiple opportunities to intervene exist [3,4,8,18,19,20,21]. These approaches may improve performance for a single intervention scenario, but they do not coordinate multiple decisions over the full case. Second, some approaches handle sequential decisions but optimize each decision point in isolation. They focus on the immediate effect of the next intervention, for instance, on the remaining processing time, without explicitly aligning interventions across decision points. This can still lead to suboptimal end-to-end outcomes with respect to the final KPI [13]. Third, other methods for sequential intervention recommendations rely on process approximations, such as Markov decision processes (MDP) or data augmentation [1,5], to train an RL agent, and the resulting policies inherit any misspecification in these approximations, also known as the reality gap, which may lead to biased or underperforming recommendations in practice [15].

To address these limitations, this paper makes two key contributions:

1. We propose *SCOPE*, a PresPM approach that combines causal learners with backward induction to learn causally grounded sequential intervention policies that are aligned across multiple decision points. For each decision point, a causal model estimates the effect of alternative interventions on the target KPI, given the observed process execution history. Backward induction then propagates the impact of later intervention decisions back to earlier ones by starting from the final decision point and recursively deriving the recommended action and its expected outcome at each preceding decision point. Operating directly on observational event logs, *SCOPE* uses causal learning to be able to identify actions rarely chosen historically but likely to enhance the target KPI, without requiring process-specific simulators or log augmentation.
2. We provide an empirical evaluation on existing synthetic data and a new semi-synthetic setup based on a real-life event log. Our code and the novel semi-synthetic benchmark are made publicly available as a reusable resource for sequential PresPM in our GitHub repository. The results show that *SCOPE* outperforms the considered baselines across a broad set of experimental settings, highlighting the importance of combining causal learning with backward induction for KPI optimization.

The remainder of this paper is structured as follows. Section 2 provides background and related work. Section 3 outlines the methodology. Section 4 presents the experiments and discussion. Finally, Section 5 concludes the paper.

2 Background

2.1 General

Current PresPM approaches are generally based on two streams of research. The first is Causal Inference (CI), which aims to estimate the effects of potential interventions from observational data and use these estimates to guide decisions. This is achieved through causal learners, which are model setups designed to predict outcomes or causal effects from observational data. For example, an S-learner fits a single model that predicts the outcome using the intervention action as an input in addition to other features. PresPM approaches typically adapt causal learners to the process context by aggregating event data for use with standard CI setups [8], or by leveraging sequential models like LSTMs [25]. [1] The second stream is RL-based, where an agent interacts with an environment that represents/approximates the real process, observes states, selects actions, and receives rewards, with the goal of learning a policy that maximizes cumulative rewards over time [18].

Similar to CI, PresPM typically relies on offline observational event logs, rather than data from controlled experiments. This approach avoids the cost and risk associated with randomized experiments, such as randomized controlled trials (RCTs) or an online RL agent exploring a real-life environment, which are often not feasible (e.g., when testing a loan assignment strategy). However, observational data introduces challenges. Because the data reflects an existing *historical decision policy* (e.g., a bank's current loan strategy), treatment/intervention assignment is not random as it would be in controlled trials. This complicates the estimation of optimal actions due to potential imbalance between treated and untreated cases or strong *confounding* factors affecting both treatment assignment and outcomes [11]. For example, if a bank offers optional financial literacy workshops (intervention) to improve repayment rates (KPI), individuals who choose to attend may already have stronger financial habits (confounder), making it difficult to isolate the workshop's true effect.

2.2 Related Work

This subsection reviews existing PresPM approaches, highlighting their key features and limitations. Table 1 summarizes how *SCOPE* compares to these methods, with the discussion below detailing the distinguishing dimensions.

Single-Interventional Approaches. Most existing PresPM methodologies focus on the impact of a single intervention scenario. For example, Bozorgi et al. [8] use causal effect estimation to identify which cases would benefit from one specific intervention decision, applying CI techniques after encoding event data into a suitable format. In their work, an example of such an intervention would be skipping a particular activity.

[1] CI typically follows causal discovery, which identifies causal relationships, while CI estimates their magnitude [25]. Most PresPM studies that optimize interventions (implicitly) adopt a known standard causal structure with time-varying confounders, interventions and an outcome.

Table 1. Comparison of PresPM methods for sequential interventions.

	Multi-interventional	**Aligned interventions across decision points**	**No process approximations required**
Why It Matters	Many business processes require a series of different intervention decisions, where each step shapes what happens next.	An intervention recommendation optimal at one decision point may be suboptimal across decision points.	Using an MDP or augmented data to approximate processes for training may misrepresent true behavior, leading to suboptimal policies.
Method			
[3,18]	×	×	×
[4,8,19,20,21]	×	×	✓
[13]	✓	×	✓
[1]	✓	✓	×
[5]	✓	✓	×
SCOPE (ours)	✓	✓	✓

Similarly, Shoush et al. [21] propose a white-box framework that integrates predictive, causal, and survival models to recommend single intervention decisions. In their case, an example intervention would be offering either multiple loan options to a client or just one. Yet, real-world business processes often require multiple (interdependent) interventions rather than a single one, as they consist of interconnected steps..

Multi-Interventional Approaches. Some PresPM approaches handle multiple interventions by recommending the next best activity to optimize a KPI. Typically, predictive models estimate KPIs for possible actions at each decision point; for instance, de Leoni et al. [13] use a transition-system to identify valid next events and select the one maximizing the predicted KPI. However, optimizing only the immediate action without considering subsequent decision points can yield suboptimal results. Consider again the revenue-maximizing marketing process with two consecutive decisions: sending a promotional email (decision point 1), then offering a discount (decision point 2). Assessing actions at decision point 1 without accounting for decision point 2 yields predictions that are *averages over the historical distribution of actions taken at decision point 2*. Consequently, the model might undervalue an email that is only highly effective when followed by a discount, ignoring the cumulative effect of action sequences. Making optimal decisions requires aligning action sequences across all decision points. Table 1 (column 2) categorizes whether methods align interventions across multiple decision points or optimize at a single point, as in [13].

A different line of work that addresses sequential interventions is presented by Abbasi et al. [1] and Branchi et al [5]. Both aim to recommend the next best activity while explicitly accounting for all possible sequences of future actions. Branchi et al. achieve this by (1) applying KMeans clustering to group process prefixes in an event log; (2) defining an MDP where states are represented by the cluster label and previous activity, rewards are defined as the average reward observed for that state in the log, and transition probabilities are obtained by replaying the log; and (3) applying offline RL (Q-learning specifically) to derive a next-best-action policy [5]. Because Q-learning optimizes cumulative return over a full case, the resulting recommendations are aligned

across decision points[2]. Abbasi et al. take a slightly different route by also applying offline RL, but combined with data augmentation [1]. They modify case timestamps and remove non-critical activities, arguing that these transformations help diversify and focus the training data. However, both approaches rely on approximations of the real process through simulation (in the form of an MDP) or data augmentation. These approximations may lead to policies that underperform in practice, a phenomenon known as the reality gap [15]. For example, in Branchi et al., important prefix information may be lost when aggregating cases prior to clustering, and KMeans may struggle with high-dimensional or categorical data. In Abbasi et al., augmentation is constrained by manually crafted rules: timestamps are only varied within 10%, and three business rules (precedence, co-occurrence, conflict) guide modifications. Some processes may require different temporal bounds or a far richer set of constraints. In such situations, the learned policy may underperform. In Table 1, the last column indicates whether a method requires an approximation (e.g., a simulation or augmentation) of the process, or can directly use the data as-is. We also note Weinzierl et al. [24], whose approach predicts the most likely future suffix and identifies similar historical cases to select actions with favorable KPIs. By only exploring the neighborhood of the (historically) most likely suffix, this method depends heavily on the historical decision policy and risks missing superior but rarely observed actions. We therefore do not pursue it further.

In summary, as shown in Table 1, current approaches exhibit a notable gap that *SCOPE* seeks to address.

Sequential Decision-Making & Causal Inference. Related to sequential PresPM are sequential decision-making approaches in CI, commonly studied under the framework of Dynamic Treatment Regimes (DTR). DTR methods aim to identify optimal treatment regimes (or policies) to maximize long-term outcomes, typically in medical settings. These methods rely on backward induction and causal reasoning to optimize long-term (medical) outcomes. For instance, in [17], the authors use Q-learning, essentially a form of backward induction, to determine the optimal treatment regime. Another approach models the DTR objective as a classification problem [26], using causal learners and a classifier to recommend the best treatment at each step. Sequential PresPM is related in spirit, but its application is not straightforward: decision-making is based on event-log prefixes, processes may exhibit variable control flow, and some decision points may be skipped. We therefore build on DTR ideas, but adapt them to the sequential PresPM setting to identify intervention recommendations that maximize process KPIs in business settings.

3 Methodology

This section introduces key definitions, notation, and a detailed explanation of *SCOPE*.

[2] Branchi et al. [5] extend their earlier work [6] by removing process-specific assumptions and thus improving generalizability, which is why we focus on it here.

3.1 Preliminaries

Definition 1 (Event, Event Log, Trace, Prefix). *An* EVENT *is a tuple* $e = (c, o, t, d, s)$*, comprising a case identifier* $c \in C$*, activity label* $o \in O$*, timestamp* $t \in \mathbb{R}^+$*, optional event-specific attributes* $d = (d_1,...,d_{m_d})$*, and static case attributes* $s = (s_1,...,s_{m_s})$*. An* EVENT LOG $L = \{e_i\}_{i=1}^{N}$ *is a collection of* N *observed events. A* TRACE $\sigma = \langle e_1,...,e_{|\sigma|} \rangle$ *is the time-ordered sequence of all events for a single case. A* PREFIX *consists of the first* l *events of a trace.*

Definition 2 (Decision point, Intervention action, Policy, Outcome). *A* DECISION POINT $k \in \{1,...,K\}$ *is a prespecified or discovered point in the process where an intervention may occur. For a case* c*, let* $\mathcal{K}^{(c)} \subseteq \{1,...,K\}$ *denote the specific decision points it actually visits, noting that some may be skipped due to variable control flow. The history available at* k *is represented by the prefix* σ_k*, encompassing all events (and interventions) preceding* k*, and its length depends on which decision points were visited. An* INTERVENTION ACTION $a_k \in A_k$ *is chosen from a feasible space* A_k*, which maps to an existing attribute in the event log (i.e., activity label or any other event-specific attribute from* s*). The historically observed action is denoted* a_k^{obs}*. A* POLICY $\pi = \{\pi_1,...,\pi_K\}$ *is a set of decision rules mapping a given prefix to an action:* $\pi_k : \sigma_k \to A_k$*. The* OUTCOME y *is the trace's final observed KPI. Consequently, an event log* L *can be encoded into the dataset:* $\mathcal{D} = \bigcup_{c=1}^{C} \bigcup_{k \in \mathcal{K}^{(c)}} \left\{ \left(\sigma_k^{(c)}, a_k^{obs,(c)}, y^{(c)} \right) \right\}$.

Objective. The goal is to find the optimal policy $\pi^{opt} = \{\pi_1^{opt},...,\pi_K^{opt}\}$, given by $\pi^{opt} = \text{argmax}_\pi \mathbb{E}\big[y(\pi) \,|\, \sigma_k\big]$, where $y(\pi)$ is the potential outcome under the policy π [16].

3.2 *SCOPE*

Below, we explain how *SCOPE* maximizes a KPI (for minimization, replace any *max* with *min*). First, we outline the theory of backward induction. Next, we describe the integration of causal learners into this procedure. Finally, we present the full *SCOPE* algorithm.

Backward Induction. We now explain how backward induction (theoretically) identifies the optimal sequential intervention policy. Following CI and PresPM literature, we begin with three standard assumptions: sequential ignorability, the stable unit treatment value assumption (SUTVA), and positivity[3]. Under these assumptions, the optimal policy π^{opt} is identifiable from observational data through backward induction, meaning it *can*, in principle, be obtained (see [17] for a detailed discussion on these assumptions and the identifiability).

Backward induction iterates through all decision points, starting from the last one (K). At K, we define the Q-function and value function as follows:

[3] Sequential ignorability: given a prefix, there are no unmeasured confounders; SUTVA: units do not affect each other's outcomes and treatments have well-defined versions; positivity: every treatment has a nonzero chance of occurring for all relevant prefixes.

$$Q_K(\sigma_K,a_K)=\mathbb{E}\big[y\,|\,\sigma_K,a_K\big];\quad V_K(\sigma_K)=\max_{a_K\in A_K} Q_K(\sigma_K,a_K). \tag{1}$$

If we then continue recursively, we define the Q-function and value function at decision point $k\in\{K-1,...,1\}$ as

$$Q_k(\sigma_k,a_k)=\mathbb{E}\big[V_{k+1}(\sigma_{k+1})\,|\,\sigma_k,a_k\big];\quad V_k(\sigma_k)=\max_{a_k\in A_k} Q_k(\sigma_k,a_k). \tag{2}$$

Then, under a perfectly known Q-function, the optimal actions and policy are given by

$$\pi_k^{\text{opt}}(\sigma_k)=\arg\max_{a_k\in\mathcal{A}_k} Q_k(\sigma_k,a_k)=a_k^{opt};\quad \pi^{opt}=\{\pi_1^{opt}(\sigma_1),...,\pi_K^{opt}(\sigma_K)\} \tag{3}$$

which concludes the backward induction.

The value function can alternatively be expressed as:

$$V_k(\sigma_k)=\mathbb{E}\big[V_{k+1}(\sigma_{k+1})+\big(Q_k(\sigma_k,a_k^{\text{opt}})-Q_k(\sigma_k,a_k^{obs})\big)\,|\,\sigma_k\big] \tag{4}$$

with $V_{K+1}=y$. This formulation, known as the regret-based form, yields the same optimal policy.

Intuitively, backward induction works by starting at the last decision point K and asking: '*Given a current prefix σ_K, what is the expected outcome of each possible action?*'. This gives us the Q-function at K. Using the standard formulation (eq. 1), the value function at K then tells us the best outcome achievable from σ_K by selecting the highest Q-value. We then move one step backward to decision point $K-1$. Here, the Q-function asks: '*If I take action a_{K-1} now for prefix σ_{K-1}, and then act optimally at K (with the new prefix σ_K resulting from a_{K-1}), what outcome can I expect?*'. This is answered by the value function at K for σ_K. The value function at $K-1$ again picks the action with the highest Q-value. We repeat this process all the way to the first decision point. In general, the Q-function answers: '*How good is it to take a specific action now, assuming we act optimally afterward?*', while the value function answers: '*How good is it to have this prefix, assuming optimal actions from here on?*'. By applying this procedure recursively, we can compute all Q-functions and thus determine the optimal policy.

Causal learners. While backward induction provides the theoretically optimal solution, applying it in practice requires estimation. This is challenging because we rely on observational event logs, and, as discussed in Section 2, observed confounders make estimation difficult. To address this, we draw inspiration from the DTR approaches in [17,26] and use causal learners to estimate the Q-function at each decision point and, consequently, the optimal action. While SCOPE still relies on function approximation to estimate Q-functions, it avoids explicit process approximations such as MDP construction or data augmentation to generate training trajectories. Causal learners are specifically designed to estimate outcomes and/or causal effects under different treatments (here, actions) using observational data [2]. These techniques try to explicitly reduce dependence on the historical decision policy (in contrast to [24], see footnote

??). In this paper, we apply three causal learners, widely adopted in CI: an S-learner, a T-learner, and an RA-learner [2], though any causal learner expected to perform well on a given dataset could be used. Each of these learners is model-agnostic: they can flexibly incorporate any suitable base predictive model (e.g., a neural network).

We adopt the regret-based formulation 4 of the value function, as it is more robust under model misspecification (e.g., incorrect model assumptions) [12]. Under this formulation, we must estimate two key components at each decision point k: the Q-function Q_k and the corresponding optimal action a_k^{opt}. Once these are obtained, computing the value function becomes straightforward. Note that, while in the theoretical setting with a perfectly known Q-function, the optimal action is always given by the action that maximizes Q_k (see eq. 3), in practical approximation settings, the optimal action may be better estimated using alternative strategies to compensate for prediction error and model uncertainty, as we do below with the RA-learner.

S-learner. The S-learner estimates Q_k using a single predictive model [2]. The model (e.g., an LSTM) takes as input the features in σ_k together with the historical action taken a_k^{obs}. At inference time, we query the model once for every possible action a_k at decision point k, producing estimates of $Q_k(\sigma_k,a_k)$. The chosen action is then simply the one with the highest predicted value.

T-learner. The T-learner builds a separate model for each possible action at decision point k [2]. Each model takes as input only the features in σ_k. At inference, we query each model to obtain the predicted outcome under its corresponding action. These predictions serve as estimates of $Q_k(\sigma_k,a_k)$, and the best action is again the one with the largest predicted score.

RA-learner. The RA-learner works in two steps [2]. First, it estimates Q_k in the same way as the S-learner. Then, it constructs a pseudo-outcome for each possible action. Intuitively, this pseudo-outcome represents the estimated causal effect of choosing action a_k on the KPI, given the observed prefix. For a given prefix, observed action a_k^{obs}, decision point k, and action of interest a_k, the pseudo-outcome is:

$$\Phi_k^{(a_k)}(\sigma_k) = \mathbf{1}\{a_k^{obs}=a_k\}(y-\hat{Q}(\sigma_k,a_k)) + \sum_{f_k\neq a_k} \mathbf{1}\{a_k^{obs}=f_k\}(\hat{Q}(\sigma_k,f_k)-y) \quad (5)$$
$$+ \sum_{f_k\neq a_k} \mathbf{1}\{a_k^{obs}=f_k\}(\hat{Q}(\sigma_k,f_k)-\hat{Q}(\sigma_k,b_k)).$$

where f_k runs over all actions other than a_k, and b_k is an arbitrary baseline action. These pseudo-outcomes (which still have the observed y in eq. 5, and thus cannot serve as estimation for new data) act as targets in a second predictive model, which is trained to estimate the causal effect of each action at decision point k. The final action estimate is the one with the largest predicted causal effect from this second model. The RA-learner thus adds an explicit second step to better estimate causal effects before choosing the final action.

Algorithm. In summary, *SCOPE* combines regret-based backward induction with causal learning to derive effective policies from observational data. In Algorithm 1, we provide an overview of the full training and inference procedure for an S-learner variant, referred to as *SCOPE-S*. Lines highlighted in green mark the steps where the Q-function and optimal action at decision point k are estimated. When adopting a T- or RA-learner, these steps are the ones that change most substantially, being replaced by the corresponding estimation procedures described above. Full algorithms for these other learners are also provided in our repository (see footnote 5).

During training, the algorithm applies backward induction, starting from the last decision point. At each decision point, it uses a causal learner (here, an S-learner) to estimate the Q-function (line 4) and determine the optimal action for every case that reached that point (line 7). It then computes the estimated value for each relevant case using a regret-based formulation (lines 8–10). If a case did not visit the current decision point (due to variable control flow in business processes), the value from the subsequent decision point is carried backward (line 13). These values serve as targets for the preceding decision point. During inference, the process moves forward. For each case, the algorithm observes the current prefix and checks whether a decision point is reached (lines 18–19). If so, it selects the model corresponding to that decision point to estimate and execute the optimal action (lines 20–21). This continues until the case is completed. [4]

4 Experiments & Discussion

This section first presents the (semi-)synthetic data and methods for comparison. Next, we describe the experimental procedure, followed by results and their implications[5].

4.1 Data

Synthetic and semi-synthetic data are widely used in causal machine learning because real-world datasets do not provide ground-truth counterfactuals. In sequential PresPM, this challenge is even stronger, since for each case we only observe the KPI under the historically executed intervention sequence, not under the many alternative sequences that could have been recommended. Real-life event logs therefore do not allow direct ground-truth evaluation of learned sequential intervention policies. To our knowledge, SimBank is the only simulator designed specifically for PresPM, which we use in our evaluation [9]. We also introduce a new semi-synthetic setup based on the BPIC17_W event log [10] and release it publicly to support further research in sequential PresPM.

SimBank. SimBank models a bank's loan application process and has been thoroughly validated. The main KPI to optimize is the total profit generated from loans in a test set. Among other options, we opt for the following two sequential interventions for this research: 1. *Choose procedure*: decide between a standard or priority procedure 2. *Set interest rate*: choose one of 3 possible interest rate levels. The effects of these interventions vary across clients, making the decisions complex.

[4] Algorithm 1: the action space can be predefined or discovered, e.g., using the transition-system in [13].

[5] The code, algorithms, and simulators are available at `https://github.com/JakobDeMoorKULstudent/SCOPE`

Algorithm 1 SCOPE-S: Training and Inference

Training (Offline)

Input: Dataset $\mathcal{D}=\bigcup_{c=1}^{C}\bigcup_{k\in\mathcal{K}^{(c)}}\{(\sigma_k^{(c)},a_k^{obs,(c)},y^{(c)})\}$, action spaces $\{A_k\}_{k=1}^{K}$

$\hat{V}_{K+1}^{(c)}=y^{(c)}\ \forall c\in\{1,...,C\}$; $\mathbf{M}=[\,]$ ▷ Initialize
for $k=K,K-1,...,1$ **do** ▷ Iterate backwards
  $\mathcal{C}_k=\{c\,|\,k\in\mathcal{K}^{(c)}\}$ ▷ Identify cases that visited k
  $\mathcal{M}_k\leftarrow$ Regress $\hat{V}_{k+1}^{(c)}$ on $(\sigma_k^{(c)},a_k^{obs,(c)})$ for all $c\in\mathcal{C}_k$ ▷ S-learner Q-function
  Append $\mathcal{M}_k$ to $\mathbf{M}$
  for each case $c\in\mathcal{C}_k$ **do**
    $\hat{a}_k^{opt,(c)}=\text{argmax}_{a_k\in A_k}\mathcal{M}_k(\sigma_k^{(c)},a_k)$ ▷ Estimated optimal action
    $\hat{Q}_k^{obs,(c)}=\mathcal{M}_k(\sigma_k^{(c)},a_k^{obs,(c)})$ ▷ Q-value of observed action
    $\hat{Q}_k^{opt,(c)}=\mathcal{M}_k(\sigma_k^{(c)},\hat{a}_k^{opt,(c)})$ ▷ Q-value of optimal action
    $\hat{V}_k^{(c)}=\hat{V}_{k+1}^{(c)}+\hat{Q}_k^{opt,(c)}-\hat{Q}_k^{obs,(c)}$ ▷ Regret-based value update
  end for
  for each case $c\notin\mathcal{C}_k$ **do**
    $\hat{V}_k^{(c)}=\hat{V}_{k+1}^{(c)}$ ▷ Carry over value if point k is skipped
  end for
end for
return M

Inference (Online)

Input: Models $\{\mathcal{M}_k\}_{k=1}^{K}$, initial prefix $\sigma_{k_{first}}^{(c)}$
while case c is not completed **do**
  if case c reaches DECISION POINT $k\in\{1,...,K\}$ **then**
    Observe current prefix $\sigma_k^{(c)}$
    $\hat{a}_k^{opt,(c)}=\text{argmax}_{a_k\in A_k}\mathcal{M}_k(\sigma_k^{(c)},a_k)$
    Execute action $\hat{a}_k^{opt,(c)}$
  end if
end while

The two decision points are highly interdependent. Selecting a standard procedure reduces costs, but increases the likelihood of a client refusal of the offer in the end. This refusal probability directly influences the optimal choice of interest rate, which must balance affordability for the client (to reduce refusals) against profitability (to cover costs, which are higher for priority procedures). The first decision thus affects both client refusal and costs, which are critical considerations in the second decision. SimBank allows us to vary the level of confounding in the training data by adjusting the % of confounded data versus full RCT data. As shown in previous studies, higher confounding typically reduces the performance of PresPM methods [8,9].

SimBPIC17 We additionally introduce SimBPIC17, a semi-synthetic simulation based on the BPIC17_W event log. In this simulation, we simplify the control-flow (by deleting 3 activities) and introduce an intervention on the existing activity 'call_incomplete_files'. At each decision point, the choice is whether to *call* the client to follow up on incomplete files or to *wait* for the client to provide the information themselves. The KPI to minimize is the total cost incurred while completing the files:

$cost_{files} = cost_{tpt} * tpt_{files} + n\_calls * cost_{call}$, where tpt_{files} is the throughput time of completing the files, which is sampled from a uniform distribution, and $cost_{tpt}$ and $cost_{call}$ are fixed constants. There is a trade-off: waiting is cheaper but increases throughput time, whereas calling is more expensive but shortens the throughput time. The (causal) effect of a call on the throughput time depends on two factors: the loan type (with 'car' and 'loan takeover' having larger effects) and the average duration of all previous activities (longer durations increase the potential time saved by calling). This dependency on average duration creates strong inter-decision point interactions: a call in decision point 1 reduces throughput time but also lowers the average duration of activities after decision point 1, which in turn reduces the effect of a call in decision point 2. To generate datasets, variables except activity labels and activity durations are sampled from the original dataset. We then define the simplified control-flow, the function determining the effect of a call, the function calculating the KPI based on the actions taken, and the bank policy, which governs whether the bank chooses to call or wait. The bank calls if the loan type is 'car' or 'loan takeover' and the average activity duration exceeds 4025 units. Unlike SimBank, SimBPIC17 makes it easy to vary the number of decision points by increasing the number of (obligatory) choices between call or wait. We mimic an observational event log by defining the bank policy, varying confounding levels in the same way as in SimBank.

4.2 Methods for comparison

We compare *SCOPE* with the multi-interventional approaches of Branchi et al. [5] and de Leoni et al. [13], each missing a key property from Table 1: [5] uses a process approximation, while [13] does not align interventions across decision points. Including both baselines allows us to examine the role of these properties, which *SCOPE* explicitly includes. We select Branchi et al. [5] as the representative method using a process approximation and exclude Abbasi et al.'s FORLAPS [1] (see Section 2). Although FORLAPS supports sequential interventions through process approximation, it is inflexible for adapting to our optimization problems. It defines MDP states solely by observed activity labels, making comparison unfair because our interventions require additional process variables. Incorporating these variables would require expanding the state space (resulting in an impractically large Q-table), redesigning the state representation, or introducing augmentation methods—changes that would fundamentally alter the original method. In contrast, Branchi et al. approximate the process as an MDP and train a Q-learning algorithm using KMeans clustering, where each state is represented by the last executed activity and the cluster label assigned to the current prefix. While the original work defines states using only control-flow information, the method can be easily extended to include additional variables by incorporating them into the clustering inputs, which is how we apply it. We refer to this approach as KMeans-Q. As our second baseline, the method of de Leoni et al. [13] represents an approach that does not align interventions across decision points. The original method trains a single model and selects actions by predicting the KPI for each possible action at every decision point, equivalent to using a single S-learner for all decision points. We adapt this approach by using a separate model per decision point, equivalent to applying an S-learner (separately) at each step.This modification is theoretically equivalent to the original method

and we implement it for three reasons: (1) a single model performed worse in our experiments; (2) it ensures a fairer comparison with SCOPE, which also uses a model at each decision point; and (3) it enables the implementation of causal learners when decision variables differ across decision points, which would not be straightforward with a single model. Because these models are applied independently, the action chosen at one decision point does not account for the impact of future actions. We refer to this approach as SEP-S, indicating the use of separate S-learners. More generally, SEP refers to any strategy in our experiments that trains separate models for each decision point. We also include a simple random policy as a baseline, and an upper bound representing the best achievable KPI. The upper bound is computed by taking, for each test case, the maximum KPI obtainable across all possible action sequences (exhaustive search).

4.3 Preprocessing & Hyperparameter tuning

Cases are truncated at the decision points, producing prefixes that end immediately before the intervention. In *SCOPE* and SEP, LSTMs use tensor encoding [23], with case-level features added at the final layer. Other models use last-event encoding for time features and aggregation encoding for other features, following [22]. Categorical features are one-hot encoded and continuous features standardized. Depending on the learner, an action feature may be included (S- or RA-learner) or not (T-learner). For KMeans-Q, we adopt the original preprocessing [5] for control-flow features and add non–control-flow features using last-event and aggregation encoding as before.

Hyperparameters are tuned via Bayesian optimization using 20% of the training set for validation. Models are then retrained on the full training set. For evaluation, we use 10,000 test cases for SimBank and 1,000 for SimBPIC17 (due to the exponential growth of sequences with decision points: $2^{n_{\text{decisionpoints}}}$). Both *SCOPE* and SEP can use any base model, tuned identically for fair comparison. In KMeans-Q, we jointly tune the KMeans and RL components using a normalized metric that combines silhouette score and average reward, allowing twice the tuning budget compared to *SCOPE* and SEP due to the dual-model setup. The original paper tuned KMeans first using clustering metrics (e.g., silhouette score) and then tuned the RL model. In our experiments, this approach performed poorly, as optimizing KMeans for clustering quality alone does not support effective intervention decisions.

4.4 Experimental Setup

To evaluate *SCOPE*, we use the *Gain*, which measures the % improvement in total KPI achieved by a method's policy over the bank's historical decision policy on the test set. We vary key simulation parameters relevant to sequential PresPM, and do this especially in SimBank, as it is thoroughly validated in [9], contains more variables, and has a more complex causal structure than SimBPIC17 (where we focus on the number of decision points). In SimBank, we adjust:

- *Confounding level δ (0.9–0.99):* Reflects real-world observational data, which is usually heavily confounded (as there is generally already a decision policy in place in business processes). Varying this helps assess whether a method is robust to biases introduced by the historical decision policy (e.g., bank policy) and whether it can still select effective actions even when those actions are rarely observed in the data.

– *Training set size (1K, 10K, 50K):* Since each model at a decision point in *SCOPE* depends on the previous one, limited data may cause estimation errors that propagate through decision points. Varying training size helps measure this effect.
– *Learners and base models:* We test different learners (S-, T-, RA-learner) and base models (XGBoost, Random Forest, LSTM/MLP) for *SCOPE* and SEP, to examine whether *SCOPE*'s advantages (due to its aligned interventions) hold consistently across learner types and underlying base model. The default is S-learner + XGBoost (e.g., for varying training size).

In SimBPIC17, the focus is evaluating performance over more than 2 decision points. We vary the number of decision points from 2 to 6 and run experiments under 3 levels of confounding (0.9, 0.95, 0.99) to assess robustness across longer decision horizons. Each setting is run using 10 different random seeds.

In our repository (see footnote 5), we provide: (1) results across different learners and base models for SimBPIC17, confirming consistency across datasets; and (2) a complexity analysis showing that, although *SCOPE* has the highest theoretical training cost, it matches SEP and outperforms the original implementation of [13] in inference complexity. While KMeans-Q is theoretically the most efficient, *SCOPE* trains significantly faster in practice, has only slightly slower inference than SEP ue to obtaining more complex models from tuning, and can even outperform KMeans-Q at inference time.

4.5 Results

Figure 1 shows method performance across confounding levels and **training sizes** on SimBank. *SCOPE* and SEP are implemented as S-learners with XGBoost (SEP thus corresponds to de Leoni et al.'s SEP-S). *SCOPE* outperforms nearly all settings, except for the (1K, 0.99) case, demonstrating its strong ability to learn sequential intervention policies. Performance generally declines with higher confounding, except for KMeans-Q at 1K and in the 50K condition, where all methods benefit from more data. As expected, larger training sets improve performance. SCOPE's advantage grows with training size, as each model uses the outputs of other models in later decision points as targets through the value function. With more data, model errors decrease and propagate less across decision points. KMeans-Q performs the worst, suggesting that approximating the process for RL while still aligning all interventions can be less effective than treating interventions independently, as SEP(-S) does.

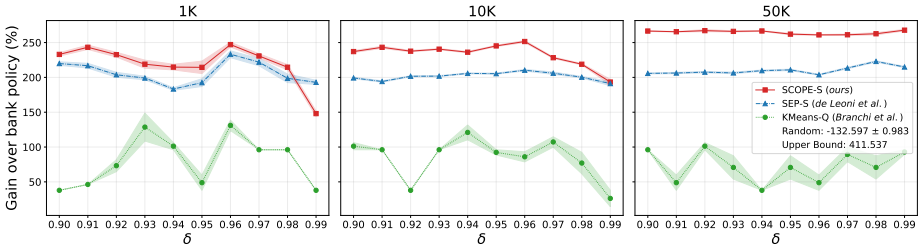

Fig. 1. Gain on SimBank across confounding levels (δ) for different training sizes. The shaded area shows one standard error over 10 iterations. *SCOPE* and Sep: S-learner with XGBoost.

Figure 2 compares *SCOPE* and SEP across confounding levels and **learner types** on SimBank (10K training cases, XGBoost). *SCOPE* consistently outperforms SEP, except for the (T-learner, 0.99) setting. Overall, the S-learner performs best, the T-learner worst, and the RA-learner's performance declines least under high confounding. Figure 3 shows performance across confounding levels and **base models** (10K training cases, S-learner)[6]. *SCOPE-S* again outperforms SEP-S. XGBoost achieves the best overall performance, while Random Forest performs worst. Both these plots suggest aligning interventions across decision points is generally more effective than treating them independently, regardless of learner or base model type.

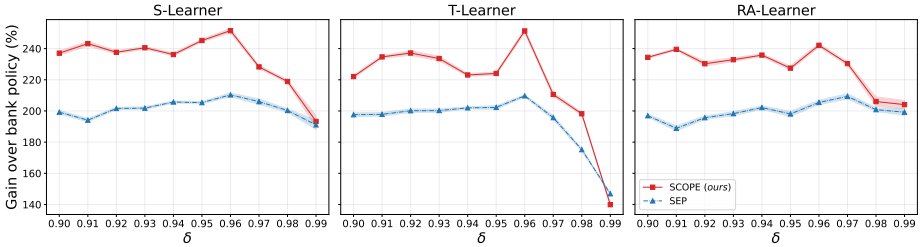

Fig. 2. Gain on SimBank across confounding levels (δ) for different learners. The shaded area shows one standard error over 10 iterations. *SCOPE* and Sep: XGBoost, trained on 10K cases.

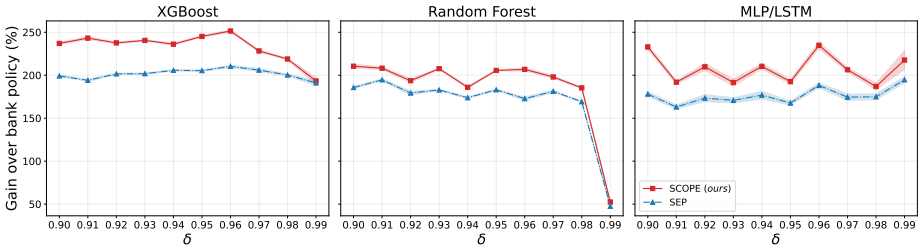

Fig. 3. Gain on SimBank across confounding levels (δ) for different base models. The shaded area shows one standard error over 10 iterations. *SCOPE* and Sep: S-learner trained on 10K cases.

Figure 4 shows method performance across different **numbers of decision points** and confounding levels on SimBPIC17 (10K training cases, *SCOPE* and SEP are again S-learners with XGBoost). *SCOPE* generally outperforms other methods, except in the (0.99, 2 decision points) setting. The performance gap between *SCOPE* and others grows with more decision points, as it introduces more interdependent decisions that *SCOPE* can effectively align. Both *SCOPE* and SEP-S gain more over the bank policy as decision points increase, but also move further from the upper bound: for *SCOPE*, due to error accumulation across models; for SEP, due to treating each decision point independently. KMeans-Q struggles with more decision points, likely because increasing the number of decision points causes the KMeans model

[6] For the right-most plot, we use an MLP for the early decision point and and an LSTM in the later decision point, where sequential structure becomes important.

to aggregate the data more heavily compared to its original form, reducing the MDP approximation quality for Q-learning.

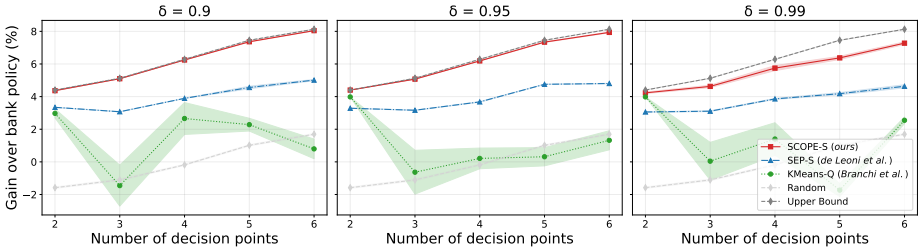

Fig. 4. Gain on SimBPIC17 across varying numbers of decision points for three confounding levels (δ). The shaded area shows one standard error over 10 iterations. *SCOPE* and Sep: S-learner with XGBoost, trained on 10K cases.

4.6 Discussion

In summary, the setup of *SCOPE* demonstrates clear advantages across a range of settings. Its performance improves with larger training sets by reducing error accumulation in backward induction and scales well with more decision points by aligning interdependent interventions, something separate optimization fails to exploit. *SCOPE* consistently outperforms separate optimization across learner types and base models. Practitioners should choose the learner–model combination expected to perform best: S-learners suffice for our datasets, but RA-learners seem more robust under strong confounding, and T-learners struggle. While RL-based approaches that approximate the entire process as an MDP (like KMeans-Q) aim for joint optimization, they can perform worse than treating decision points independently. *SCOPE* does rely on the predictive accuracy of underlying models: a highly unpredictable KPI can lead to strong error accumulation. However, this challenge affects all methods, and *SCOPE* still likely retains its advantage by leveraging observational event logs directly and considering the interdependent decision point structure.

Limitations. *SCOPE* models sequential dependencies within individual process cases but assumes no interference between interventions across different cases, as required by the SUTVA assumption (Section 3). Similarly, sequential ignorability may not always hold in business processes. Despite these assumptions—common to most PresPM approaches—our experiments show that *SCOPE* can still identify effective sequential policies under realistic levels of confounding typical in business processes. Moreover, *SCOPE* theoretically increases training complexity by requiring a separate predictive model and backward induction at each decision point. However, as shown in our repository (see footnote 5), practical results differ: despite being theoretically most demanding, *SCOPE* trains significantly faster than KMeans-Q. During inference, *SCOPE* matches SEP's theoretical complexity (and is more efficient than the original implementation of [13]), with only slightly longer runtimes in practice due to obtaining more complex tree models during hyperparameter tuning.

5 Conclusion

We introduce *SCOPE*, a method that combines backward induction with causal learning to optimize sequential interventions in business processes. Using a series of experiments, including a newly developed semi-synthetic simulator to support further research in sequential PresPM, we show that *SCOPE* outperforms existing sequential PresPM methods. Existing approaches either handle each intervention independently or rely on approximating the process to train an RL agent, both of which can limit their effectiveness.

Future work could explore extending *SCOPE* to account for interference between cases or relax the sequential ignorability assumption, while still leveraging backward induction and causal learning. This might involve exploring causal methods for handling interference [7] and using instrumental variables to address hidden confounders [14]. Additionally, the computational load could be reduced by trying to adjust our method into a single end-to-end neural framework, similar to [11] but focused on backward induction instead.

References

1. Abbasi, M., Khadivi, M., Ahang, M., Lasserre, P., Lucet, Y., Najjaran, H.: An innovative data-driven and adaptive reinforcement learning approach for context-aware prescriptive process monitoring (2025)
2. Acharki, N., Lugo, R., Bertoncello, A., Garnier, J.: Comparison of meta-learners for estimating multi-valued treatment heterogeneous effects. ICML'23, JMLR.org (2023)
3. Bozorgi, Z.D., Dumas, M., Rosa, M.L., Polyvyanyy, A., Shoush, M., Teinemaa, I.: Learning when to treat business processes: Prescriptive process monitoring with causal inference and reinforcement learning (2023)
4. Bozorgi, Z.D., Teinemaa, I., Dumas, M., Rosa, M.L., Polyvyanyy, A.: Prescriptive process monitoring for cost-aware cycle time reduction. In: ICPM (2021)
5. Branchi, S., Buliga, A., Francescomarino, C.D., Ghidini, C., Meneghello, F., Ronzani, M.: Recommending the optimal policy by learning to act from temporal data (2023)
6. Branchi, S., Di Francescomarino, C., Ghidini, C., Massimo, D., Ricci, F., Ronzani, M.: Learning to act: A reinforcement learning approach to recommend the best next activities. In: BPM Forum. Springer (2022)
7. Caljon, D., Van Belle, J., Berrevoets, J., Verbeke, W.: Optimizing treatment allocation in the presence of interference. EJOR **328**(2), 620–632 (2026)
8. Dasht Bozorgi, Z., Teinemaa, I., Dumas, M., La Rosa, M., Polyvyanyy, A.: Prescriptive process monitoring based on causal effect estimation. Information Systems **116** (2023)
9. De Moor, J., Weytjens, H., De Smedt, J., De Weerdt, J.: Simbank: From simulation to solution in prescriptive process monitoring. In: BPM Forum. Springer (2026)
10. van Dongen, B.: Bpi challenge (2017), `https://data.4tu.nl/articles/dataset/BPI_Challenge_2017/12696884`
11. Hess, K., Frauen, D., Melnychuk, V., Feuerriegel, S.: IGC-net for conditional average potential outcome estimation over time. In: ICLR (2026)
12. Huang, X., Choi, S., Wang, L., Thall, P.F.: Optimization of multi-stage dynamic treatment regimes utilizing accumulated data. Statistics in Medicine **34**(26) (2015)
13. Leoni, M.d., Dees, M., Reulink, L.: Design and evaluation of a process-aware recommender system based on prescriptive analytics. In: ICPM (2020)

14. Lousdal, M.L.: An introduction to instrumental variable assumptions, validation and estimation. Emerging Themes in Epidemiology **15** (2018)
15. Ma, S., Flanigan, K.A., Bergés, M.: Bridging the reality gap in digital twins with context-aware, physics-guided deep learning. Journal of Computing in Civil Engineering (2026)
16. Rubin, D.B.: Estimating causal effects of treatments in randomized and nonrandomized studies. Journal of Educational Psychology **66**(5), 688–701 (1974)
17. Schulte, P.J., Tsiatis, A.A., Laber, E.B., Davidian, M.: Q- and a-learning methods for estimating optimal dynamic treatment regimes. Statistical Science (2014)
18. Shoush, M., Dumas, M.: Prescriptive process monitoring under resource constraints: A reinforcement learning approach. Künstliche Intelligenz **39**, 119–140 (2025)
19. Shoush, M., Dumas, M.: Intervening with confidence: Conformal prescriptive monitoring of business processes (2022)
20. Shoush, M., Dumas, M.: Prescriptive process monitoring under resource constraints: A causal inference approach. In: Process Mining Workshops. Springer (2022)
21. Shoush, M., Dumas, M.: White box specification of intervention policies for prescriptive process monitoring. Data & Knowledge Engineering (2025)
22. Teinemaa, I., Dumas, M., Rosa, M.L., Maggi, F.M.: Outcome-oriented predictive process monitoring: Review and benchmark. ACM Trans. Knowl. Discov. Data **13**(2) (Mar 2019)
23. Verenich, I., Dumas, M., Rosa, M.L., Maggi, F.M., Teinemaa, I.: Survey and cross-benchmark comparison of remaining time prediction methods in business process monitoring. ACM Trans. Intell. Syst. Technol. **10**(4) (Jul 2019)
24. Weinzierl, S., Dunzer, S., Zilker, S., Matzner, M.: Prescriptive business process monitoring for recommending next best actions. In: BPM Forum. Springer (2020)
25. Weytjens, H., Verbeke, W., De Weerdt, J.: Timed process interventions: Causal inference vs. reinforcement learning. In: BPM Workshops. Springer (2024)
26. Zhan, Z., Liu, Z., Lin, C., Yi, D., Liu, J., Yang, Y.: Censored c-learning for dynamic treatment regime in colorectal cancer study. Annals of Applied Statistics (2025)